

L09: Advanced Deep Learning for Computer Vision

CSci 560 Deep Learning w/ Python (Chollet) Ch. 9

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

L09.1 Three Essential Computer Vision Tasks

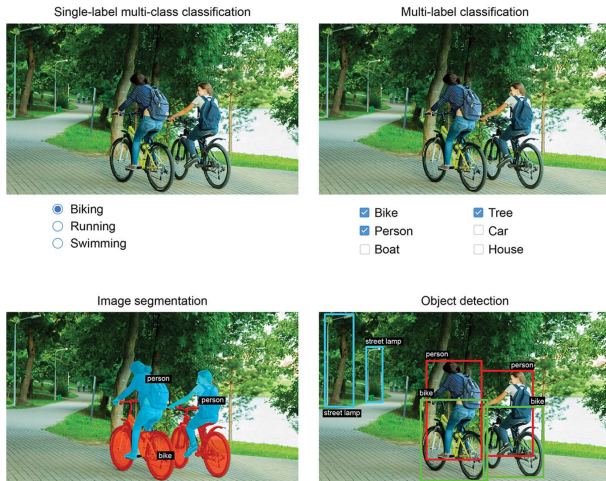
CSci 560: Deep Learning w/ Python (Chollet) Ch. 9.1

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

The Three Main Computer Vision Tasks



- **Image classification** goal is to assign one or more labels to whole image.
- **Image segmentation** goal is to “segment” or “partition” an image into different areas.
- **Object detection** goal is to draw **bounding boxes** around objects of interest and classify.

Figure 1: The three main computer vision tasks: classification, segmentation, detection. (Chollet, 2021, pg.239)

L09.2 An Image Segmentation Example

CSci 560: Deep Learning w/ Python (Chollet) Ch. 9.2

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

Semantic vs. Instance Segmentation

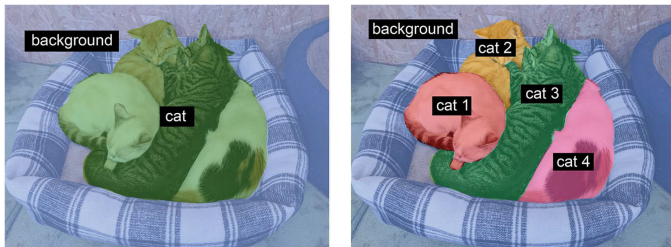


Figure 2: Semantic segmentation vs. instance segmentation.
(Chollet, [2021](#), pg.241)

- **Semantic segmentation** each pixel is independently classified.
- **Instance segmentation** seeks not only to classify image pixels by category, but also to parse out individual object instances.

L09.3 Modern Convnet Architecture Patterns

CSci 560: Deep Learning w/ Python (Chollet) Ch. 9.3

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

Modern Convnet Architecture Patterns

- So far we've discussed the basics of how image classification and segmentation work.
- The convnets used in real-world problems are not quite so simple and are more varied.
- Still lacking essential mental models and thought processes that enable experts to put together state-of-the-art models.
- To bridge the gap, this section is about **architecture patterns** for more complex use cases.

Modern Convnet Architecture Patterns

- A model's **architecture** is the sum of the choices that went into creating it:
 - Which layers to use
 - How to configure them
 - what arrangement to connect them
- These choices define the **hypothesis space** of your model:
 - the space of possible functions that gradient descent can search over
 - parameterized by the model's weights
- Like feature engineering, the hypothesis space encodes your **prior knowledge** about the problem at hand and its possible solutions.

Model Architecture: difference between success and failure

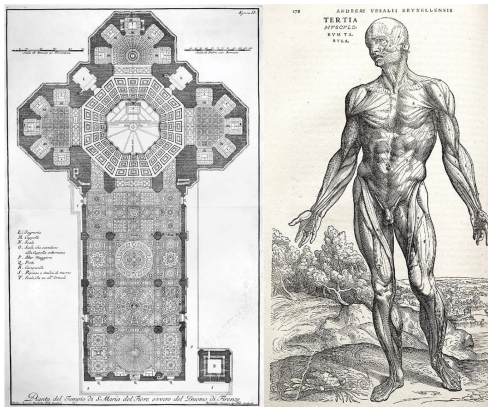
- A good model architecture is one that:
 - **reduces the size of the search space**
 - or otherwise **makes it easier to converge to a good point of the search space**
- Model architecture is all about **making the problem simpler** for gradient descent to solve.
- Gradient descent is a pretty stupid search process, it needs all the help it can get.
- There isn't much in the way of actual science, but as in any engineering discipling, there are known best practices that you can follow when you are beginning out.

Model Architecture Best Practices

Some convnet architecture best practices we will discuss:

- **residual connections**: fix vanishing gradients by retaining noiseless versions of the input information
- **batch normalization**
- **separable convolutions**

Modularity, Hierarchy and Reuse



The general approach to manage a complex system:

- Structure amorphous soup of complexity into **modules**
- Organize the modules into a **hierarchy**
- Start **reusing** the same modules in multiple places (this is a type of **abstraction**)

That's the MHR formula:
modularity-hierarchy-reuse

Figure 3: Complex systems follow a hierarchical structure and are organized into distinct modules, which are reused multiple times. (Chollet, 2021, pg.249)

Software Architecture and Deep Learning Architecture

If you're a software engineer, you're already keenly familiar with these principles: an effective codebase is one that is modular, hierarchical, and where you don't reimplement the same thing twice.

- Deep learning is simply the application of this recipe to continuous optimization via gradient descent:
 - Take an optimization technique (gradient descent over a continuous function space)
 - Structure search space into modules (layers)
 - Organized into a deep hierarchy (could be just a stack, though we will see others later)
 - Where you reuse whatever you can (e.g. convolutions are all about reusing the same information in different spatial locations).

Deep Learning Architecture Goals

- Deep learning model architecture is primarily about making clever use of modularity, hierarchy and reuse.
- Most convnets feature pyramid-like structures **feature hierarchies**.
- Deeper hierarchies are intrinsically good because they encourage feature reuse, and therefore abstraction.
- For example, the VGG16 architecture we used before is structured into repeated “conv, conv, max pooling” blocks.

VGG16 Architecture for example

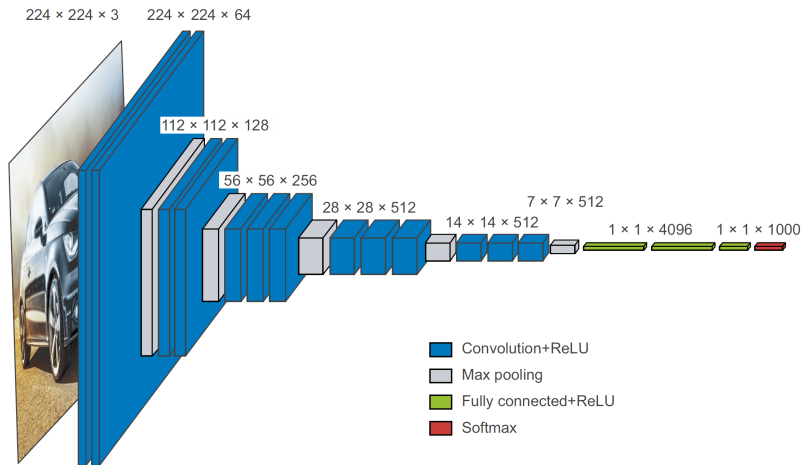


Figure 4: The VGG16 architecture: note the repeated layer blocks and the pyramid like structure of the feature maps. (Chollet, [2021](#), pg.250)

Vanishing Gradients Problem

- Backpropagation in a sequential deep learning model is a chain of functions
$$y = f_4(f_3(f_2(f_1(x))))$$
- The game is to adjust the parameters of each function in the chain based on the error recorded on the output of f_4 (the loss of the model).
- Each successive function in the chain introduces some amount of noise.
- If your function chain is too deep, this noise start overwhelming gradient information, and backpropagation stops working.

When your model won't train at all because of this, this is known as the **vanishing gradients** problem.

Residual Connections: Fixing vanishing gradients

- To fix vanishing gradients: retain a noiseless version of the information contained in the previous input
- Easiest way is to use **residual connections**
 - just add the input of a layer or block of layers back to its output.
 - act as an **information shortcut** around destructive or noisy blocks

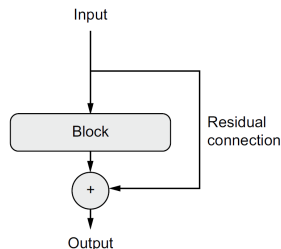


Figure 5: A residual connection around a processing block. (Chollet, [2021](#), pg.252)

Implementing a Residual Connection

In practice, in the Keras functional API, a residual connection would look something like:

```
# some input tensor
x = keras.Input()

# save a pointer to this original input, this is called the residual
residual = x

# this computation block can potentially be destructive or noisy,
# and thats fine
x = block(x)

# add the original input to the layer's output, the final output will thus
# always preserve full information about the original input
x = add([x, residual])
```



Residual Block where the Number of Filters changes

Adding the input back to the output block implies that output has same shape as input. For convolutional or maxpooling layers this is not the case. Use a 1×1 Conv2D layer with no activation to linearly project the residual to the desired output shape.

```
inputs = keras.Input(shape=(32, 32, 3))
x = layers.Conv2D(32, 3, activation="relu")(inputs)

# set aside the residual
residual = x

# we want residual around this layer, the layer increase the number of output
# filters from 32 to 64
x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)

# the residual only had 32 filters, so use 1 x 1 Conv2D to project it to correct shape
residual = layers.Conv2D(64, 1)(residual)

# now the block output and the residual have the same shape
x = layers.add([x, residual])
```



Case where the Target Block includes a Max Pooling Layer

Need to use strides in the residual project to match any downsampling caused by a max pooling layer.

```
inputs = keras.Input(shape=(32, 32, 3))
x = layers.Conv2D(32, 3, activation="relu")(inputs)

# set aside the residual
residual = x

# This is the block of two layers around which we create residual connection
# it included a 2x2 max pooling layer
x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
x = layers.MaxPooling2D(2, padding="same")(x)

# use strides=2 in the residual project to match the downsampling
residual = layers.Conv2D(64, 1, strides=2)(residual)

# now the block output and the residual have the same shape
x = layers.add([x, residual])
```



Batch Normalization

- **Normalization** is a broad category of methods that seek to make different sample be more similar to each other
 - Most common is centering data and normalizing standard deviation to 1.0
 - This makes assumption that data follows a normal distribution
- Data normalization may be useful after every tensor transformation
 - Even if data entering is normalized, it won't be anymore after tensor transformation for layers
 - Could normalizing intermediate activations help?
- **Batch normalization** does this
 - In keras can be applied as a layer `BatchNormalization`
 - Adaptively normalize data even as the mean and variance change over time.

In practice the main effect of batch normalization appears to be that it helps with gradient propagation, and thus allows for deeper networks.

Example Keras BatchNormalization Layer

The BatchNormalization layer can be used after any layer - Dense, Conv2D, etc.

```
x = ..  
x = layers.Conv2D(32, 3, use_bias=False, activation="relu")(x)  
x = layers.BatchNormalization()(x)
```

Both Dense and Conv2D layers have a **bias vector**. Because normalization step takes care of centering, the bias vector is no longer needed when using batch normalization.

It is a matter of debate, but some recommend putting the output activation after the normalization step, e.g.

```
# don't use bias weights, and no output activation  
x = layers.Conv2D(32, 3, use_bias=False)(x)  
x = layers.BatchNormalization()(x)  
# applying relu activation after normalization in Keras API  
x = layers.Activation("relu")(x)
```



Depthwise Separable Convolutions

- **Depthwise separable convolution** SeparableConv2D layer performs a spatial convolution on each channel of input independently, before mixing output channels via pointwise convolution.
- Drop-in replacement for Conv2D: make model smaller, leaner and probably improve performance (win-win-win)

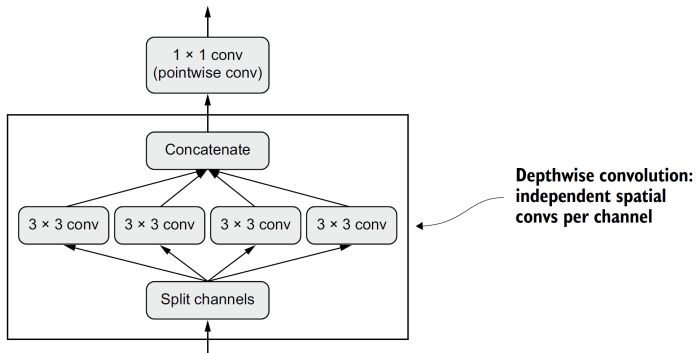


Figure 6: Depthwise separable convolution: a depthwise convolution followed by a pointwise convolution. (Chollet, 2021, pg.257)

Depthwise Separable Convolutions

In much the same way convolution relies on assumption that patterns are not tied to specific locations, depthwise separable convolution assumes that **spatial locations** in intermediate activations are **highly correlated** but **different channels** are **highly independent**.

Depthwise separable convolution requires significantly fewer parameters and involves fewer computations compared to regular convolution.

While having comparable representational power.

It results in smaller models that converge faster and are less prone to overfitting.

Convenet Architecture Best-practices Summary

- Your model should be organized into repeated **blocks** of layers, usually made of multiple convolution layers and a max pooling layer.
- The number of filters in your layers should increase as the size of the spatial feature maps decreases.
- Deep and narrow is better than broad and shallow.
- Intoducing residual connections around blocks of layers helps you train deeper networks (avoids vanishing gradients)
- It can be beneficial to introduce batch normalization layers after your convolution layers (though reason why these work remain controversial, may also just be helping reduce vanishing gradients).
- It can be beneficial to replace Conv2D layers with SeparableConv2D layers, which are more parameter-efficient.

L09.4 Interpreting what Convnets Learn

CSci 560: Deep Learning w/ Python (Chollet) Ch. 9.4

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2025

Chollet, F. (2021). *Deep learning with python* (second). Manning.